

Computer vision

Local descriptors

Image description

Image description -> features

Various approaches:

- Colour features
 - Shape features
 - Texture features
 - Edge features
-
- Global features
 - Local features

! Combination of all these approaches

Basic concept of local description

Motivation

- Object detection -> Pattern recognition (Obsolete?)
- Image registration
- 3D reconstruction -> Wide Multiple Baseline Stereo
- Sparse optical flow
- ...more



Local image descriptors

Partially possible:

- Occlusions of objects

- Changes of camera view position

- Rotation, scale invariance

- Intra category variations

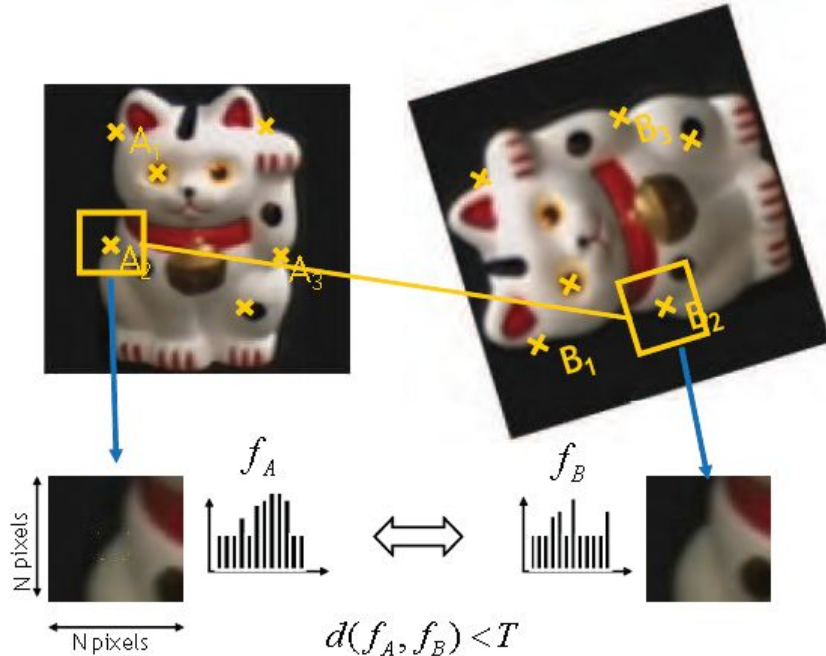
<http://vgg.fiit.stuba.sk/kniha/>

Local description (detectors + descriptors)

What we need:

- Local description which includes:
 - Local detectors** - Detection of interesting points (IP)
-key-points
 - Local descriptors** - Description of surrounding area of
the key-points in the form of a descriptor.
- Matching algorithm

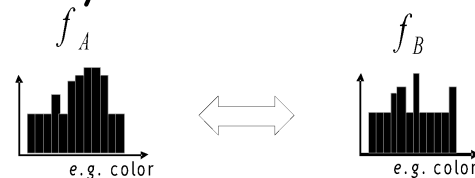
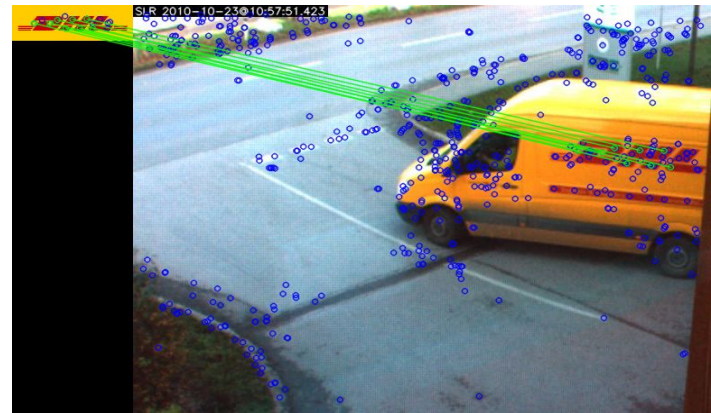
An illustration of the recognition procedure with local features



Scale- and rotation-invariant!

Template detection (registration) using local descriptors

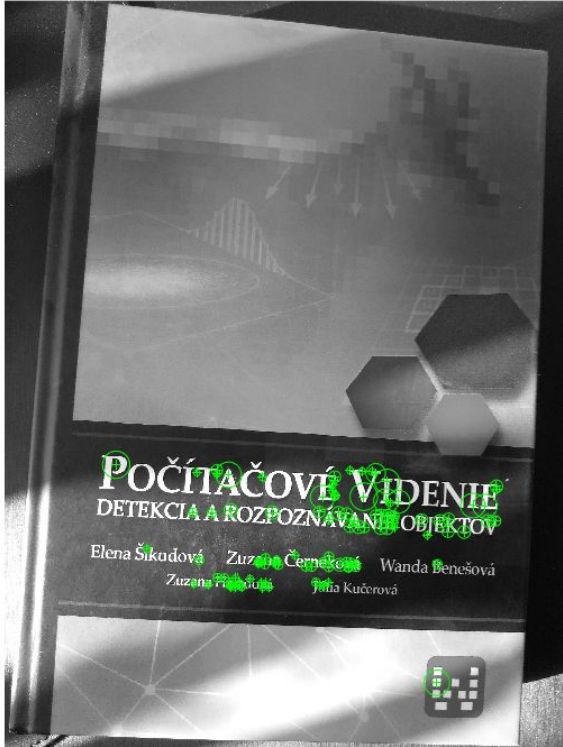
1. Find a set of distinctive key-points in the template and in the image
 - Define a region around each keypoint (orientation and scale invariance?)
 - Compute a local descriptor from the region
2. Match local descriptors
3. Filter the correspondences (...RANSAC)
4. Calculate homography



$$d(f_A, f_B) < T$$

1.-3. Key points detection and description

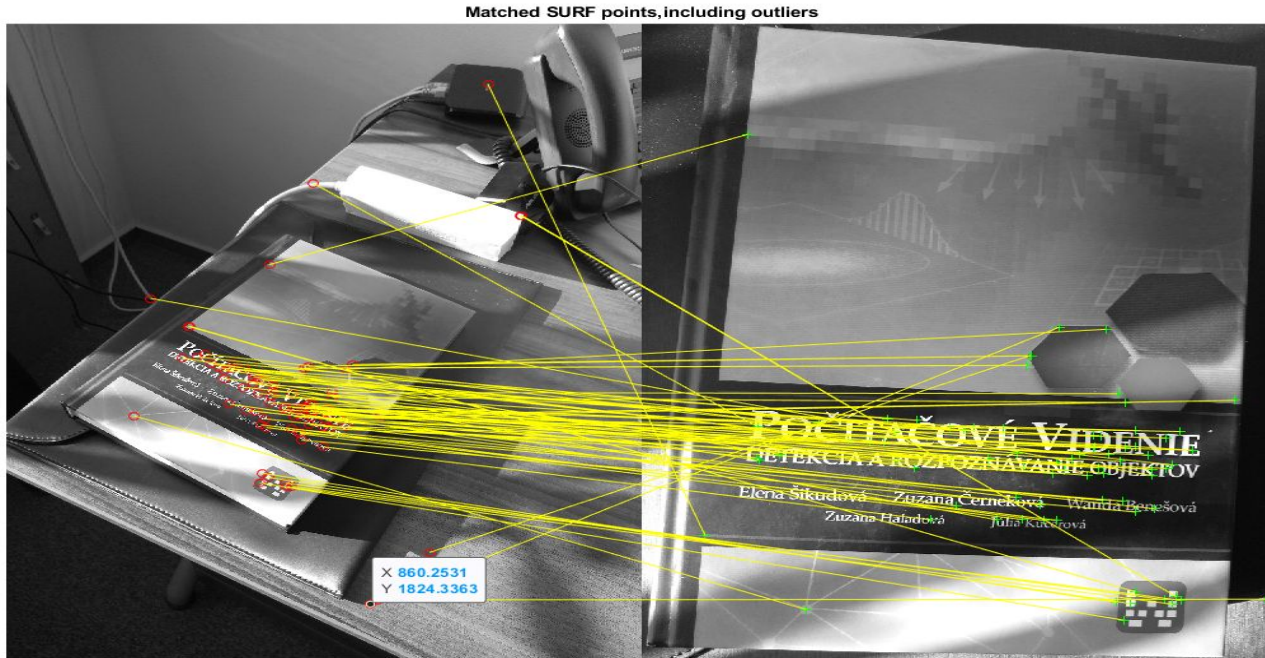
100 Strongest Feature Points from Box Image



300 Strongest Feature Points from Scene Image



4. Match local descriptors



5. Filtering of inlier/outlier

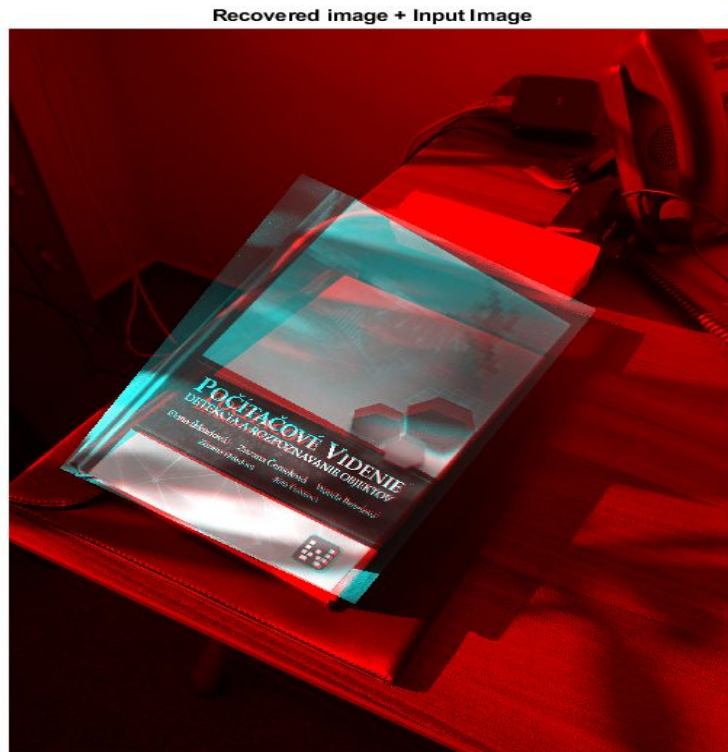
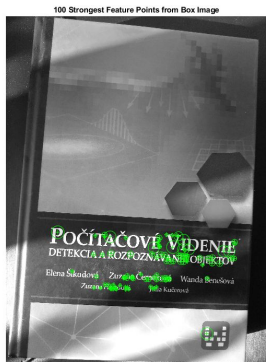
Matched inlier points



6. Calculate homography

Result after homography transform
Visualisation:

Template projection – blue channel
Original Image – red channel

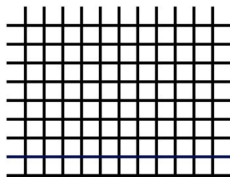


Local detectors

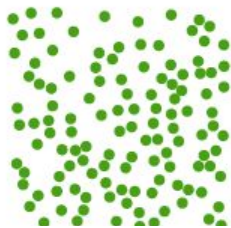
- Harris corner detector
- Shi-Tomasi detector
- Hessian point detector
- Fast key-point detector

Detection of interest points and regions - keypoints localization strategies

Regular



Random



Interest point detector



Corner detectors

Hessian point detector

Harris corner detector

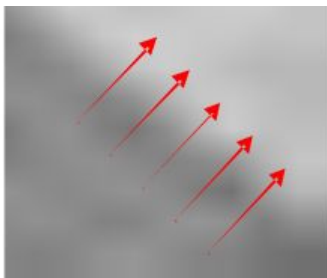
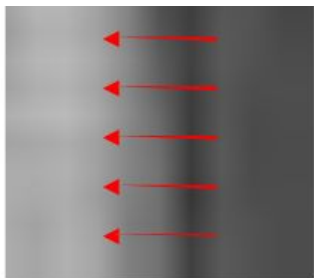
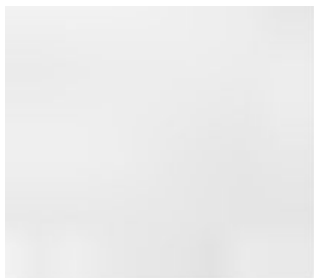
Shi-Tomasi detector („Good features to track“)

Corner detector

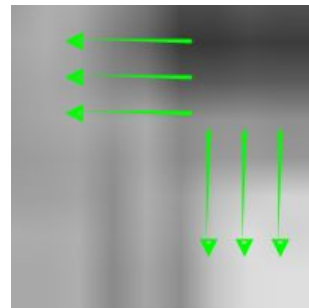
Intuition:

Search for local neighborhoods where the image content has **two main directions**.

undistinguished patches:

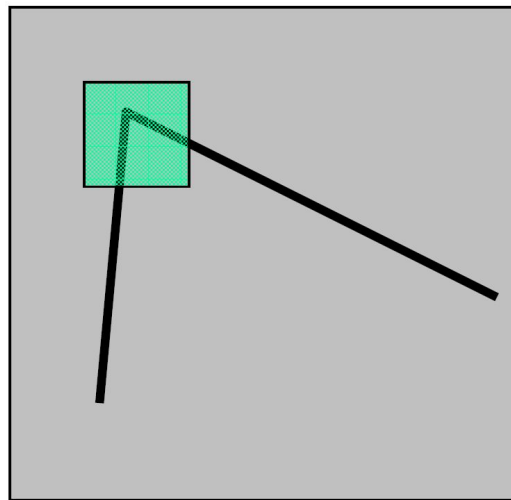


distinguished patch:



Basic idea of a Local detector

- We should easily recognize the point by looking through a small window
- Shifting a window in any direction should give a large change in intensity



Hessian matrix

The Hessian matrix is a square matrix of second-order partial derivatives of a scalar-valued function, or scalar field.

$$\mathbf{H}_f = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}.$$

Hessian point detector - mathematical background

Search for strong partial derivatives in two orthogonal directions

Main idea:

The key point is detected as a local extrema of the determinant of the Hessian matrix (calculated in each point in the image)

$$Hessian(I) = \begin{bmatrix} I_{xx} & I_{xy} \\ I_{xy} & I_{yy} \end{bmatrix}$$

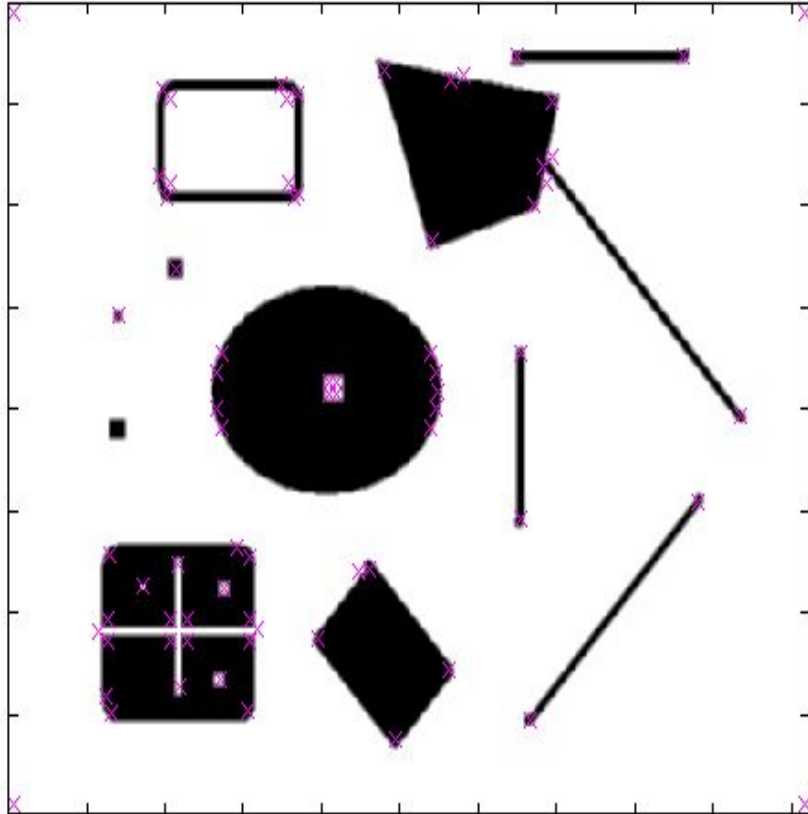
I_{xx} is second partial derivative in the a direction x
 I_{yy} is second partial derivative in the a direction y
 I_{xy} is mixed partial derivative in the x and y directions.

$$\text{Determinant } \det(I) = I_{xx} * I_{yy} - I_{xy}^2$$

Hessian point detector - calculation steps

- computes the partial derivatives I_{xx} , I_{xy} , I_{yy} (values of Hessian matrix) for each image point in the image -> then calculate image containing the **determinant values of the Hessian matrix**
- search for the points **where the determinant values are above** the pre-defined **threshold θ** .
 - search include **non-maximum suppression** on this **image** using a 3×3 window.
 - the search window is swept over the entire image, keeping only pixels whose value is larger than the values of all 8 immediate neighbours inside the window

Hessian point detector example



Effect: Responses mainly on corners and strongly textured areas.



Harris corner detector

Harris corner detector

->derived from second-moment matrix M

The Harris Corner detector is just a mathematical way of determining which windows produce large variations when moved in any direction.

R – response signal :

- result of calculation of large local variations derived from image derivatives (differences) in x and y directions respectively. (...this can be easily calculated using Sobel filter)

Corner detection: if $R > \text{threshold}$

Harris corner detector

This slide will not be included in the exam

Define second-moment matrix M of the image I :

$$M = \sum_{(x,y) \in W} \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix} = \begin{bmatrix} \sum_{(x,y) \in W} I_x^2 & \sum_{(x,y) \in W} I_x I_y \\ \sum_{(x,y) \in W} I_x I_y & \sum_{(x,y) \in W} I_y^2 \end{bmatrix}$$

I_x and I_y

are image derivatives (differences) in x and y directions respectively.

(Can be easily calculated using Sobel filter)

$w(x, y)$

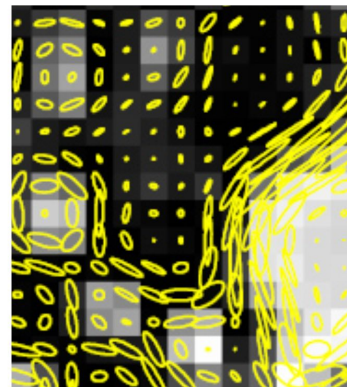
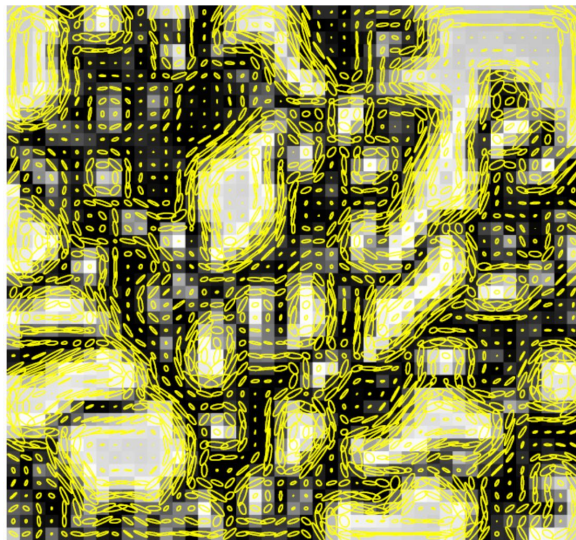
is a function of the window which is determined as the weighted sum (Gaussian core),

Second Moment Matrix

This slide will not be included in the exam

The second moment matrix tells us how quickly the image changes and in which directions.

Visualizing Second Moment Matrix:



Technical note: M is often best visualized by first taking inverse, so long edge of ellipse goes along edge

Harris corner detector

For detecting of corners are important the eigenvalues of matrix M , namely (λ_1, λ_2) .

(λ_1, λ_2) reflect the variance in two main directions.

If the **variance in both directions is large** (eigenvalues λ_1, λ_2 are large enough)

-> the point is located at the corner (corner point), and if it is large only one value, lies on the edge (one direction).

Math recap: eigenvalue and eigenvector

This slide will not be
included in the exam

Definition:

The basic equation (A : square matrix,
vector x : column vector)

$$Ax = \lambda x.$$

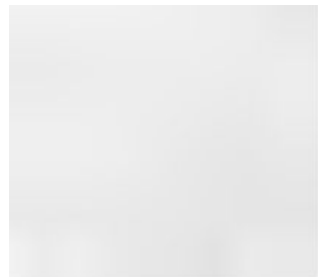
The number λ is an eigenvalue of A .

Corner detection using eigenvalues of matrix M

This slide will not be included in the exam

$(\lambda_1, \lambda_2) = \text{eigenvalues}(M)$

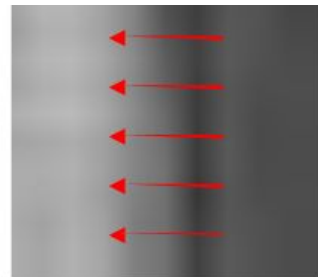
“Flat” region



λ_1 and λ_2 are small;

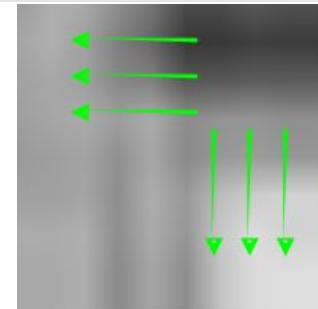
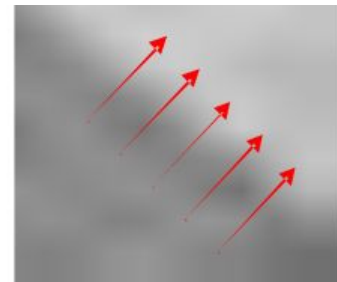
“Edge”

$\lambda_1 \gg \lambda_2$



“Edge”

$\lambda_2 \gg \lambda_1$



“Corner”

λ_1 and λ_2 are large,
 $\lambda_1 \sim \lambda_2$;

Harris corner detector

This slide will not be
included in the exam

The Harris Corner detector is just a mathematical way of determining which windows produce **large variations when** moved in any direction.

if $R > \text{Threshold}$ -> the point is detected as a key point

$$R = \lambda_1 \cdot \lambda_2 - k (\lambda_1 + \lambda_2)^2 = \det(M) - k \text{trace}(M)^2$$

where k is an empirically determined constant

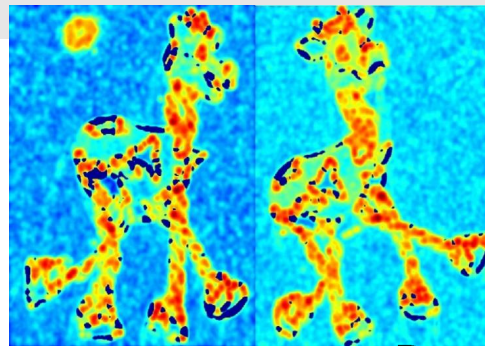
$$\det(M) = \lambda_1 \cdot \lambda_2$$

$$\text{Trace}(M) = \lambda_1 + \lambda_2$$

Harris corner detector example

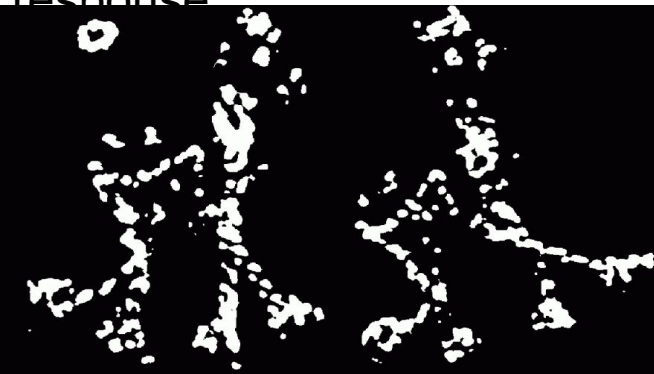


Compute corner
response R :



Take only the points of local maxima of R

Find points with large corner
response:



Shi-Tomasi detector („Good features to track“)

Harris metric has been simplified in Article [J. Shi a C. Tomasi 94]:

The corner detector searches for key points where matrix M has :

Both eigenvalues $> Th$: $\lambda_1 > Th$ and $\lambda_2 > Th$

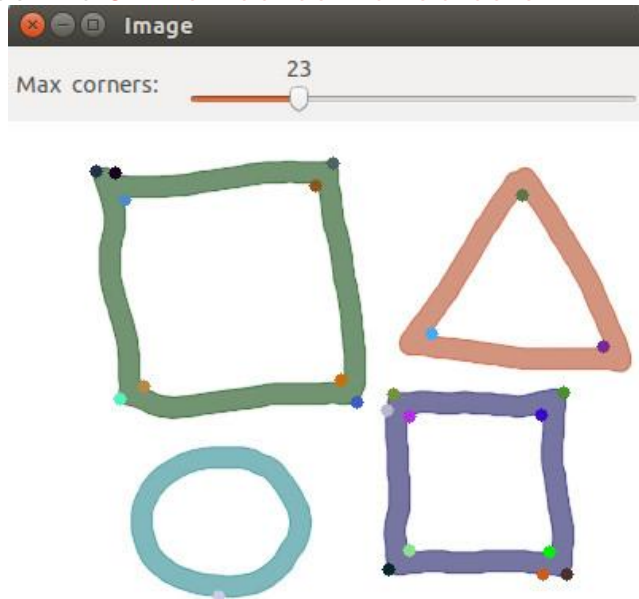
-> corresponding to two dominant orientations

(λ_1, λ_2) are eigenvalues(M) M is second-moment matrix M

Shi-Tomasi detector („Good features to track“)

OpenCV implements an improved version of this corner detector. It is called the *Shi-Tomasi corner detector*

https://docs.opencv.org/3.4/d8/dd8/tutorial_good_features_to_track.html



J. Shi a C. Tomasi. „Good features to track“. *Computer Vision and Pattern Recognition, 1994. Proceedings CVPR '94., 1994 IEEE Computer Society Conference on.* June 1994, str. 593–600.

FAST keypoint detector

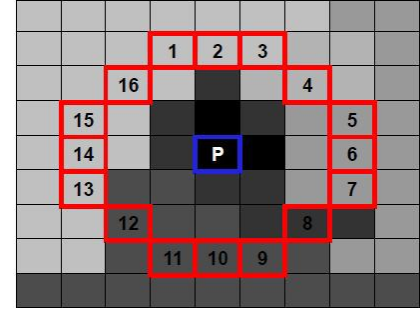
FAST keypoint detector example

Example of detection of interesting points using FAST method:

P is origin point,

in the are given by distance $r = 3$ exist 11 of adjacent points (1-7, 13-16) which intensity of is higher than $P + t_{\text{intensity}}$.

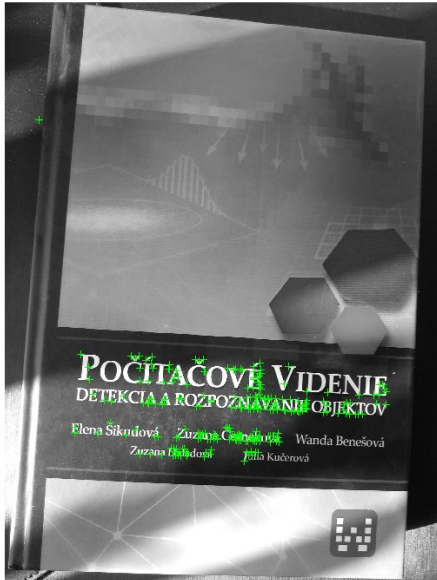
Point P is detected as an interesting point.



FAST keypoint detector example

template

Correspondences?

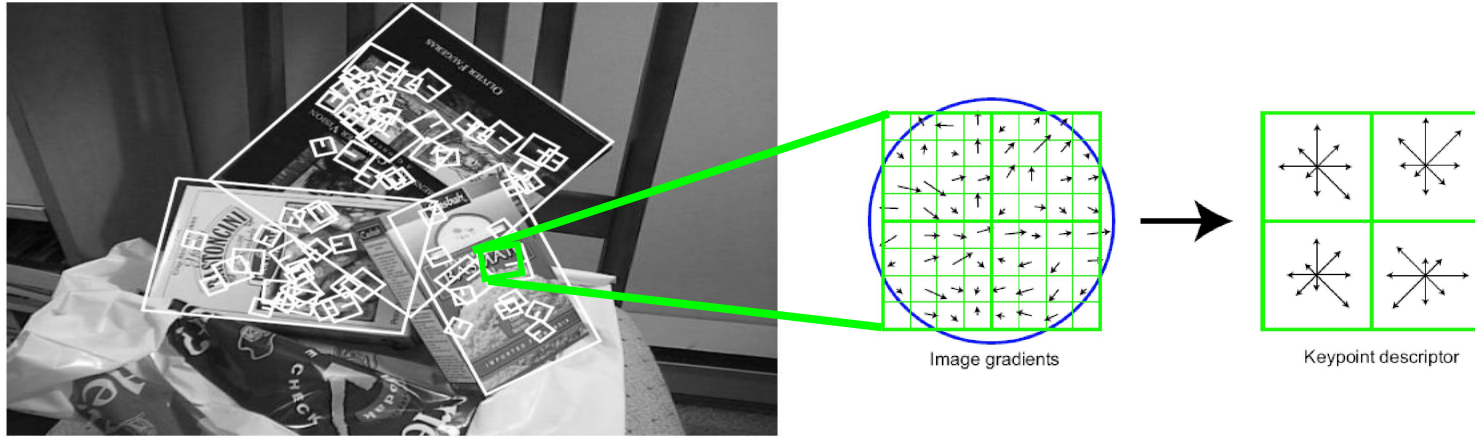


SIFT detector

SIFT includes detector + descriptor
(patent)

SIFT (detector + descriptor)

Scale-invariant feature transform



Histogram of oriented gradients

Captures important texture information

Robust to small translations / affine deformations

SIFT detector

Each key point specifies:

- 2D coordinates x, y -> based on Hessian point detector
- Scale -> Scale invariance
- Orientation -> Orientation invariance

SIFT detector includes Automatic scale selection

The principle of automatic scale selection :

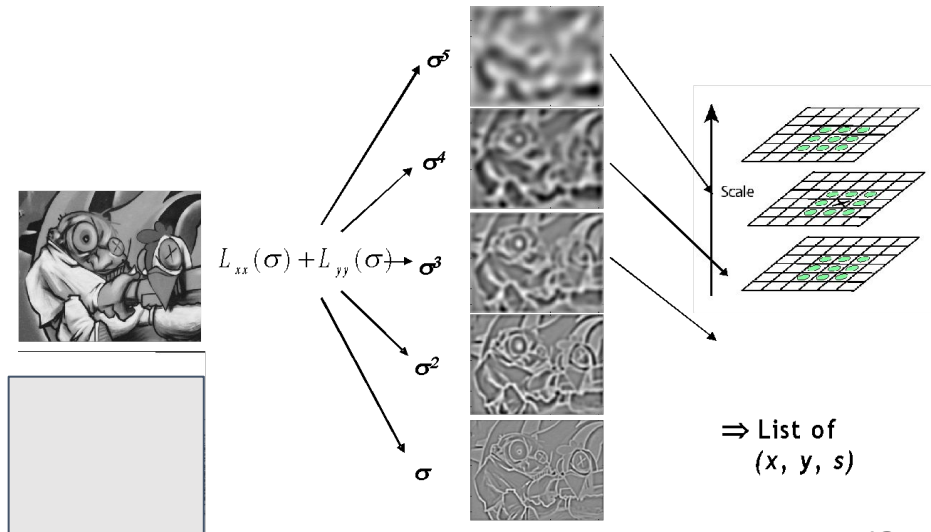
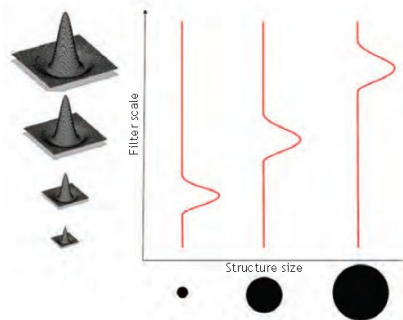
For a given a key-point location,
we evaluate a scale-dependent signature function
on the key-point neighbourhood and **find the max
value on the scale.**

SIFT detector - scale selection using Laplacian of Gaussian (LOG)

Automatic scale selection:

Response of the LOG filter is large if the size of the input „blob“ is corresponding to the filter size.

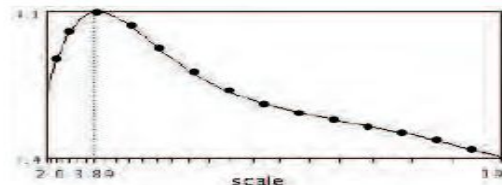
? Which scale in scale space of Laplacian-of- Gaussian gives the Local Maximum?
-> this is the detected scale



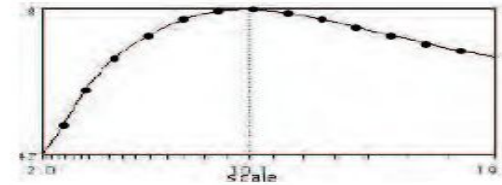
SIFT detector includes Automatic scale selection

corresponding neighbourhood sizes can be determined by searching for scale-space extrema of the signature function independently in both images.

FIGURE FROM
Krystian Mikolajczyk



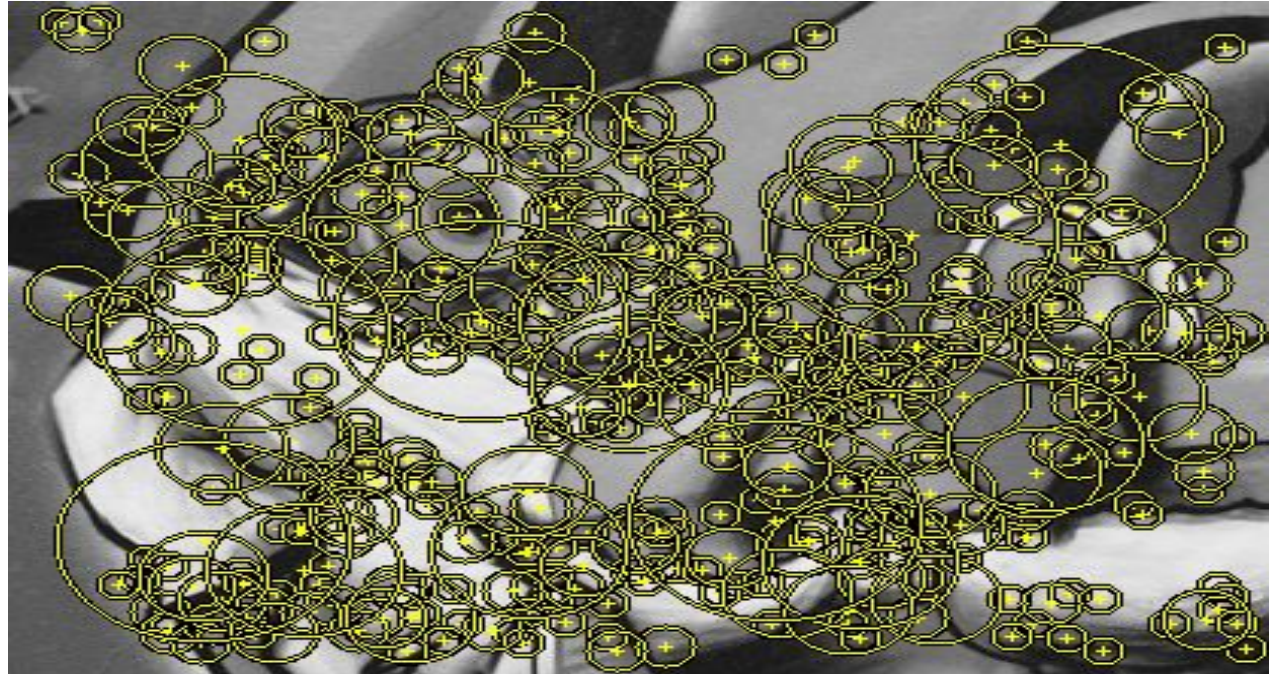
$$f(I_{i_1...i_m}(x, \sigma))$$



$$f(I_{i_1...i_m}(x', \sigma'))$$

SIFT point detector example of detected key points

Different size of key points - patches used for descriptor

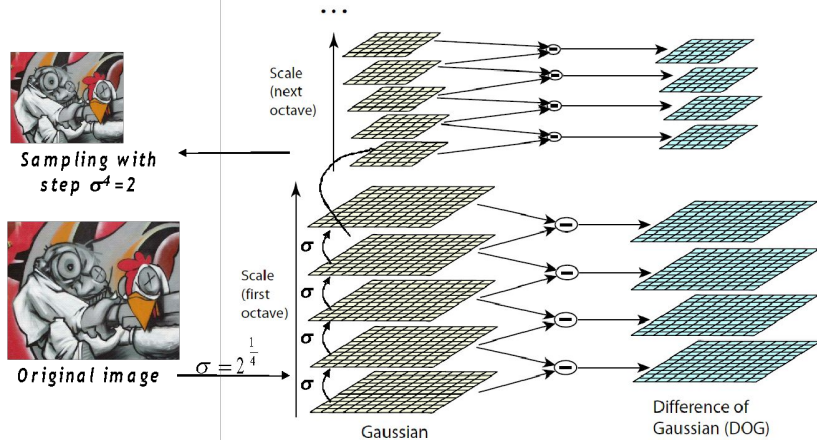
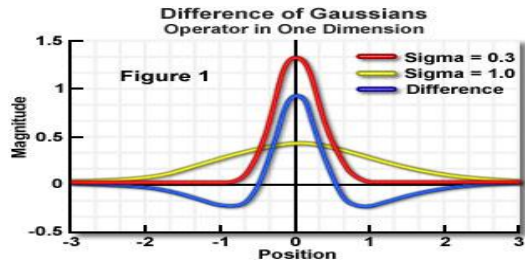
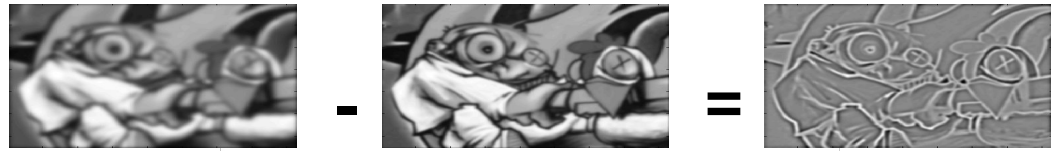


Computation of DOG instead of LOG

-> more efficient computing

Difference of Gaussian images (DOG) are approximately equivalent to the Laplacian of Gaussian (LOG)

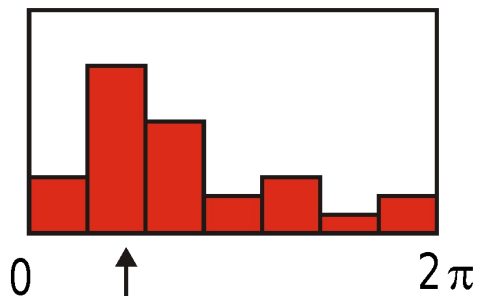
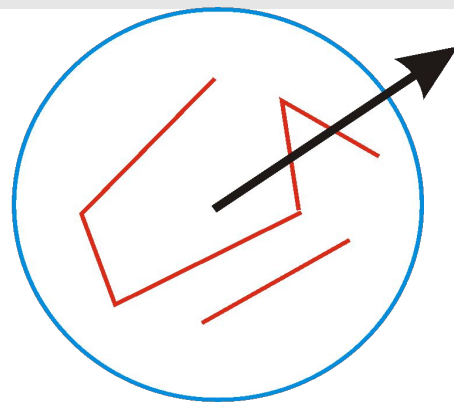
-> calculation is more efficient using DOG



Orientation-invariance of SIFT

- Create histogram of local gradient directions computed at selected scale
- Assign canonical orientation at peak of smoothed histogram

-> Dominant orientation is detected

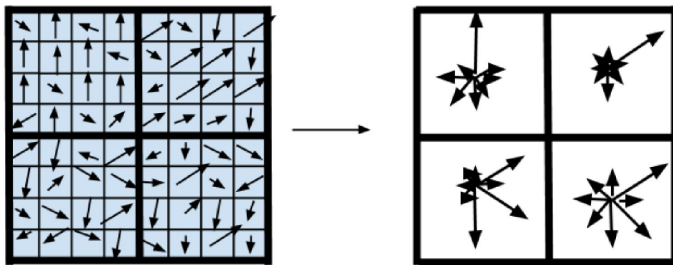


SIFT descriptor – signature vector

SIFT includes detector + descriptor

Creating SIFT Signature

- Thresholded the image gradients (magnitude $> Th$)
 - -> only the gradients above the threshold create a histogram
- Main idea similar with HOG (Histogram of Gradients)-> Calculate histograms of gradients

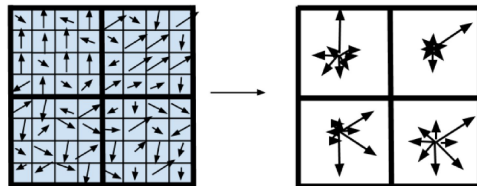


Creating SIFT Signature

Calculate histograms of gradients:

- histograms are computed of samples in a 16×16 region around the key-point
- each histogram contains samples from a 4×4 subregion of the original neighbourhood region.
- The descriptor is a vector of all the values of these histograms.

(Since there are $4 \times 4 = 16$ histograms each with 8 bins the vector has 128 elements.)



SIFT original paper and citation

Distinctive image features from scale-invariant keypoints

Authors David G Lowe

Publication date 2004/11/1

Journal International journal of computer vision

Volume 60

Issue 2

Pages 91-110

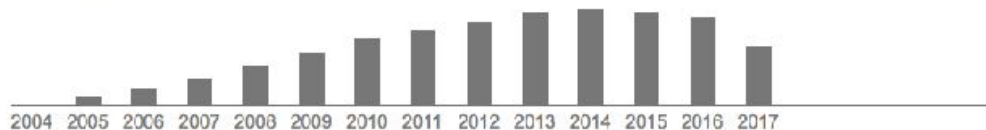
Publisher Springer Netherlands

Description This paper presents a method for extracting distinctive invariant features from images that can be used to perform reliable matching between different views of an object or scene. The features are invariant to image scale and rotation, and are shown to provide robust matching across a substantial range of affine distortion, change in 3D viewpoint, addition of noise, and change in illumination. The features are highly distinctive, in the sense that a single feature can be correctly matched with high probability against a large database of features from ...

Total citations Cited by 43944

Number of citation 2020

61621

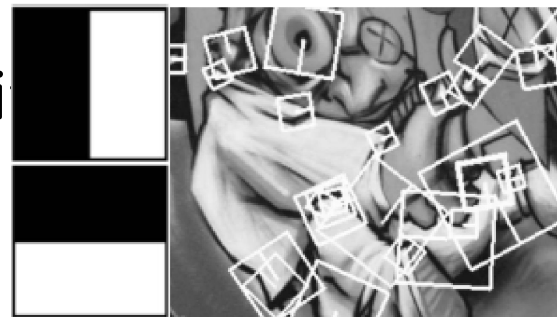


SURF

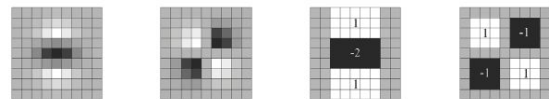
(patent)

SURF

- Fast approximation of SIFT idea
- Efficient computation by 2D box filters & integral images
6 times faster than SIFT
- Equivalent quality (?) for object identification



[Bay, ECCV'06], [Cornelis, CVGPU'08]

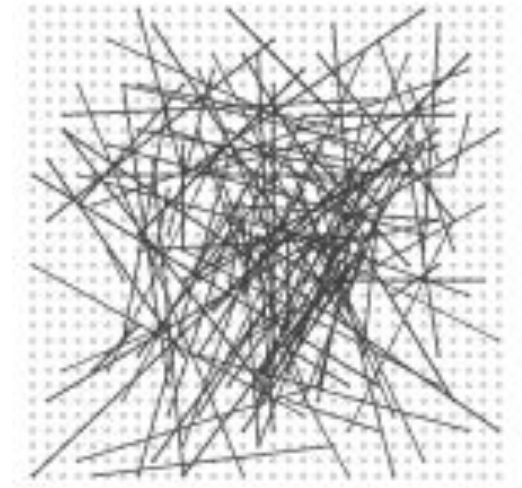


Binary local descriptors

Binary Robust Invariant Elementary Features : BRIEF

The BRIEF descriptor is constructed using a set of binary tests between pixels. around a key-point.

0100011101010001000010001010



Binary descriptor - BRIEF

BRIEF a binary descriptor which describes a patch of the image, where each bit corresponds to a binary test result to the intensity difference.

We can imagine the descriptor pattern like the small sticks disseminated on the patch. Then we look at the two ends of each stick and compare their value of intensity.

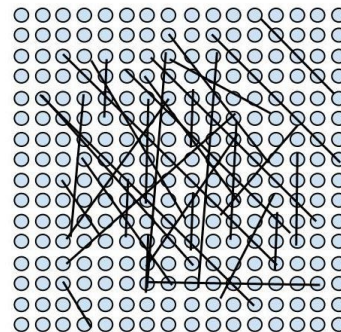
The resulting descriptor length will contain

1 if:

$I(\text{start}) > I(\text{end})$

0 if:

$I(\text{start}) \leq I(\text{end})$



Oriented BRIEF : ORB

ORB uses the descriptor modification BRIEF -> BRIEF with rotation.

The rotation of the detected interesting point is extracted using a **center of gravity** of the intensity of the surrounding area.

The method assumes that the vector of the interesting point in the center of gravity (Center of gravity of intensity) induces rotation.

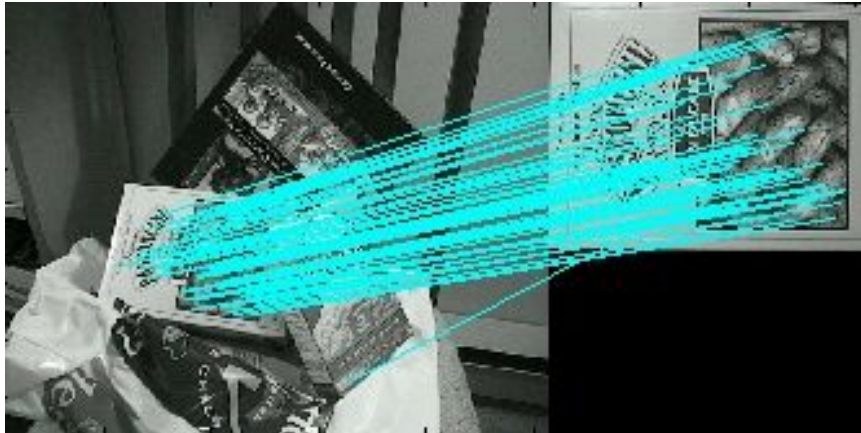
Matching of local features

Matching Local Features -> Estimating the alignment

Similarity search:

Euclidean distance , Hamming distance ...

to the nearest neighbour - > search corresponding pairs



2D Geometric Transformation

2D Affine Transformation

- Linear transformations
 - Translations
- Parallel lines remain parallel

2D Projective Transformation

- Affine transformations, and
 - Projective warps
- Parallel lines do not necessarily remain parallel

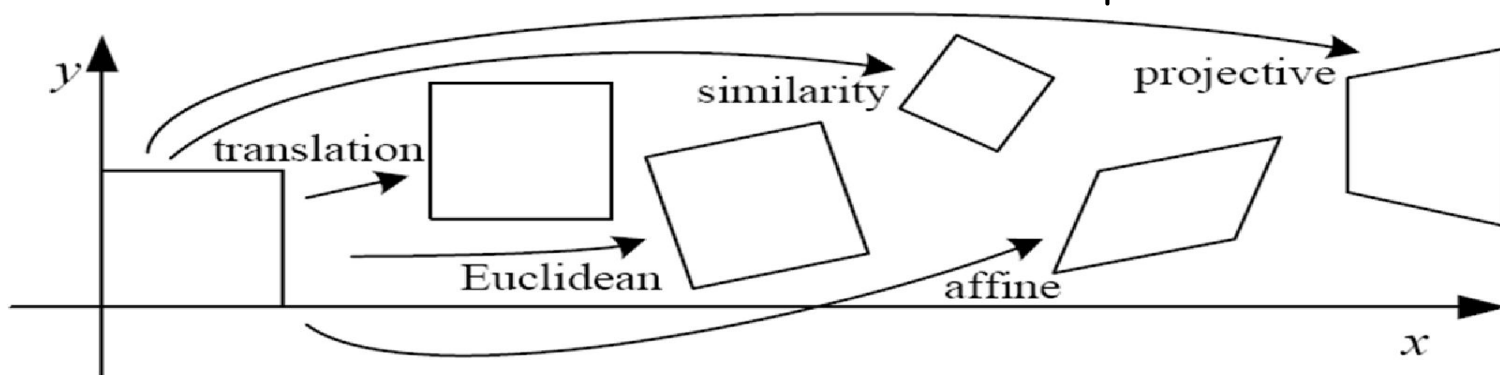
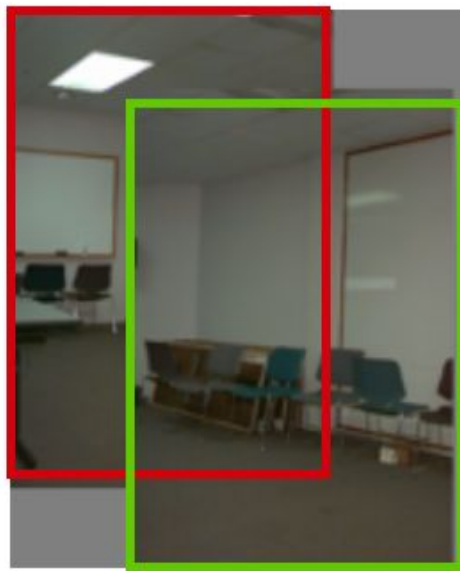


Image Warping

Geometric Transformation

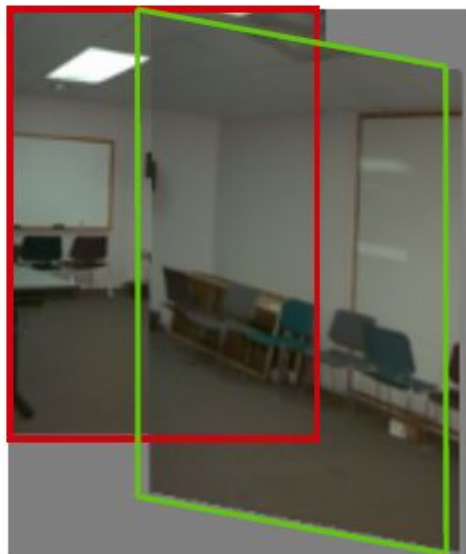
Translation

2 unknowns (2×1)



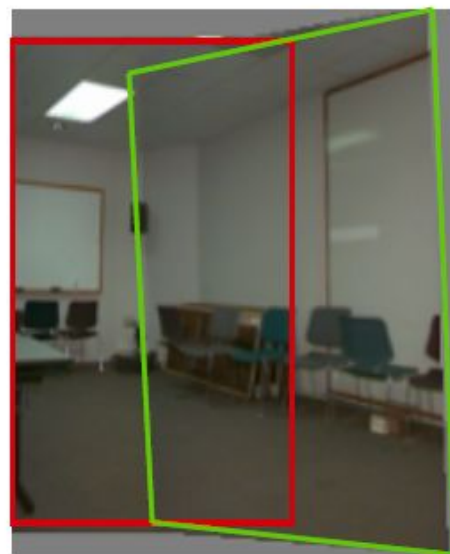
Affine trans.

6 unknowns (2×3)



Projective trans.

8 unknowns (2×4)



2D Homography matrix

$x = (u, v, 1)$ point in original image and

$x' = (u', v', 1)$ point in target image

• A homography matrix is a 3 by 3 matrix $M = \begin{bmatrix} m_{11} & m_{12} & m_{13} \\ m_{21} & m_{22} & m_{23} \\ m_{31} & m_{32} & m_{33} \end{bmatrix}$

• The homography relates the pixel coordinates in the two images if

$$x' = M x$$

• When applied to every pixel the new target image is a warped version of the original image

Matching key-points -> feature pairs

Alignment:

We are looking for a set of matching **feature pairs** ("correspondences")

Minimum set are 4 corresponding **feature key-point pairs**

-> then we can fit the parameters of the geometric transformation

Direct method (brute force)

- Define the Distance function $\text{Dist}(F_i, F_j)$
We need to calculate a distance between the feature key-points F_i and F_j .

Float features: Euclidean distance

Binary features: Hamming distance

The Hamming Distance is a number used to denote the difference between two binary strings. (XOR)

- Search over all feature pairs
calculate $\text{Dist}(F_i, F_j)$ for all combination
Find corresponding target feature to each original feature and vice versa -> select the minimum $\text{Dist}(F_i, F_j)$

Filtering inliers/outliers

Random sample consensus (RANSAC)

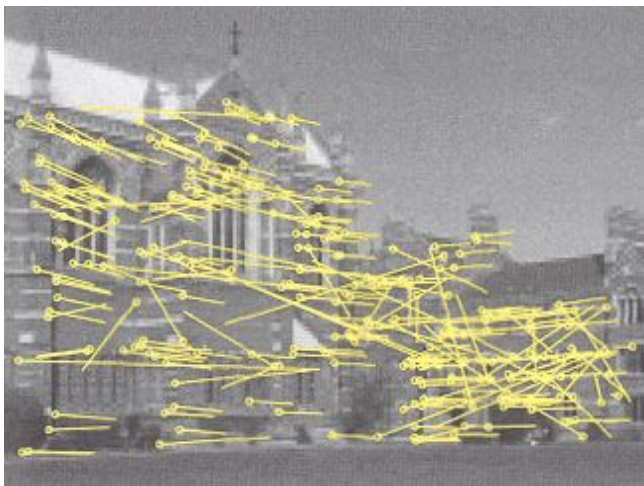
Main idea of the method:

1. Randomly select 4 corresponding feature pairs
 1. Compute homography matrix from this randomly selected corresponding points
 1. Using that homography, measure error on inliers/outliers
 - Inliers are in a well-overlapped match position after the warp using this homography matrix. Outlier not.
- Typically accepted if min 50% are inliers.
 - Output best one found

Example: Finding Feature Matches

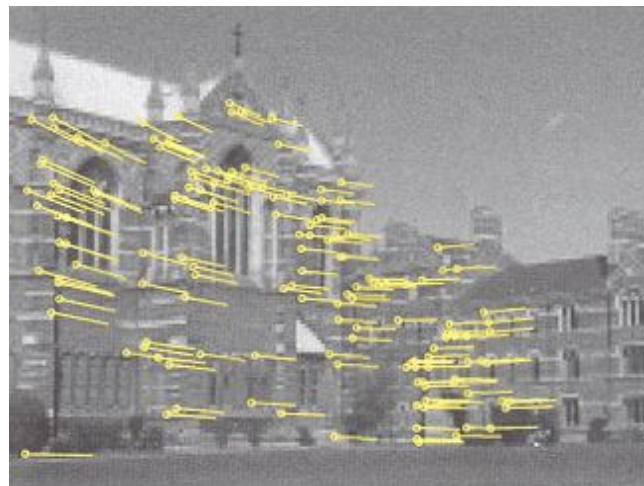
Find best stereo match within a square search window
(here: 300 pixels²)

before RANSAC



from Hartley & Zisserman

after RANSAC



K. Grauman, B. Leibe

Slide credit: David Lowe

RANSAC loop (iterative method)

RANSAC loop:

- Select four feature pairs (at random)
- Compute homography H (exact)
- Compute **inliers**
- Keep largest set of inliers
- Re-compute least-squares H estimate on all of the inliers

Visual words

Bag of words - BoF

Bag of visual words

Bag of visual words (BoVW) is one of the pre-deep learning methods used for building image embeddings.

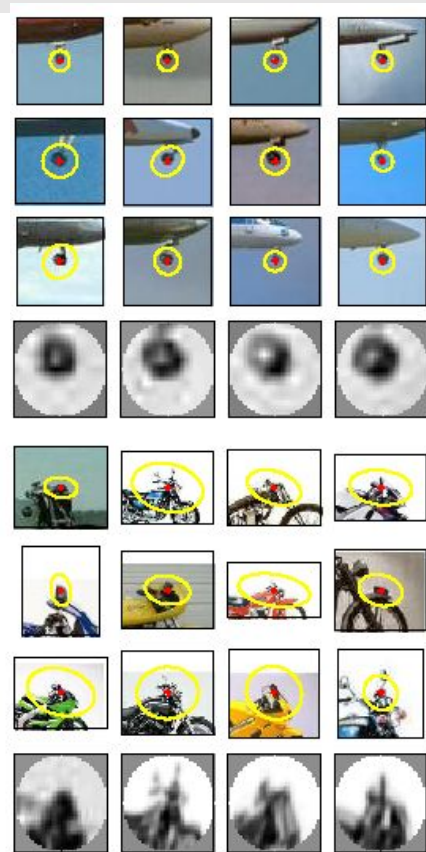
Visual words - main idea

Visual words: quantization of feature space

Clustering of local features (SIFT, SURF...) into the clusters – visual words

(...typically more than 1000 clusters)

-> a local key point can be then described only by the corresponding cluster index



Visual words - Example

Example: each group of patches belongs to the same visual word

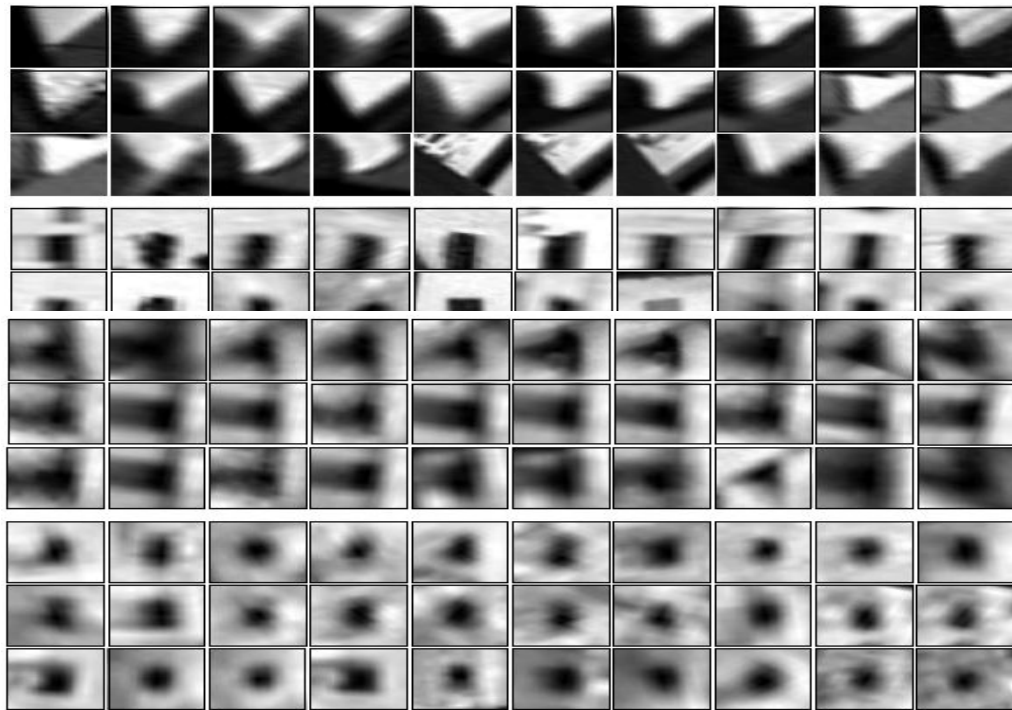
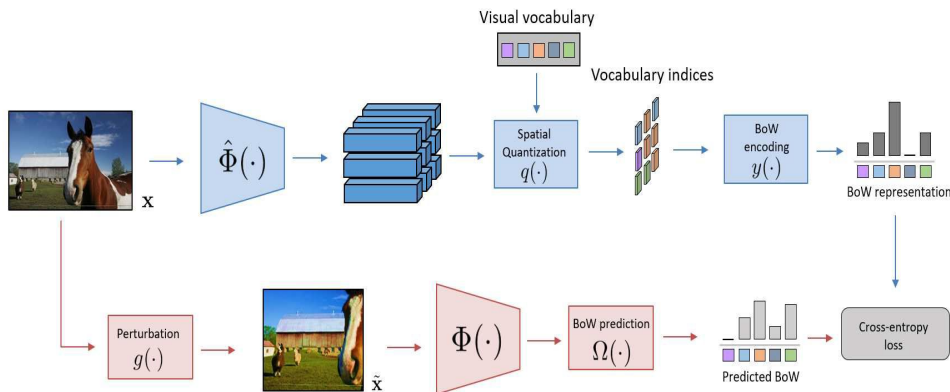


Figure from Sivic & Zisserman, ICCV 2003

Learning Representations by Predicting Bags of Visual Words

- inspired by the success of NLP methods - Transformers
- self-supervised approach based on spatially dense image descriptions called visual words
- quantization the feature maps
- prediction of the histogram of visual words of an image (i.e., its Bag-of-Words representation)



1. First train a feature extractor $\Phi(\cdot)$ for a self-supervised task, e.g. rotation prediction.
2. Then compute a visual vocabulary from feature vectors computed from Φ feature maps and compute the corresponding image level BoW vectors. These BoW vectors will serve as ground truth for the next stage.
3. In the second stage we perturb images with $g(\cdot)$ and send them as input to a second network $\Phi(\cdot)$.
4. The BoW prediction module $\Omega(\cdot)$ processes $\Phi(\cdot)$ feature maps to predict BoW vectors corresponding to the original non-perturbed images.
5. Both $\Phi(\cdot)$ and $\Omega(\cdot)$ are trained jointly with cross-entropy loss.
6. The feature extractor $\Phi(\cdot)$ is further used for downstream tasks.